CHƯƠNG 3 · BÀI 2

3.2. Promise & Async/Await

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu xử lý bất đồng bộ và callback
> - Sử dụng Promise với then, catch
> - Viết mã bất đồng bộ dễ đọc bằng async/await

1. Bất đồng bộ

Một số tác vụ tốn thời gian như gọi API, đọc file, hẹn giờ không nên chặn chương trình. JavaScript xử lý chúng bất đồng bộ: tác vụ chạy nền và báo kết quả khi xong. Cách cũ là dùng callback, nhưng lồng nhiều callback gây khó đọc (callback hell).

2. Promise

Promise là đối tượng đại diện cho một tác vụ bất đồng bộ. Promise có ba trạng thái: pending (đang chờ), fulfilled (hoàn thành thành công) và rejected (thất bại). Sau khi hoàn thành thành công, Promise ở trạng thái fulfilled và trả kết quả qua `.then()`; lỗi được bắt bằng `.catch()`.

```
const cho = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

cho(1000)
  .then(() => console.log("Sau 1 giây"))
  .catch((loi) => console.error(loi));
```

3. async và await

Cú pháp `async`/`await` giúp viết mã bất đồng bộ trông giống mã đồng bộ: từ khoá `await` tạm dừng hàm `async` cho tới khi Promise được giải quyết. Dùng `try...catch` để bắt lỗi.

```
async function taiDuLieu() {
  try {
    await cho(500);
    console.log("Đã tải xong");
  } catch (loi) {
```

```
    console.error("Lỗi:", loi);
  }
}
```

4. Chạy song song

`Promise.all([p1, p2])` chờ tất cả Promise hoàn thành và trả về mảng kết quả; nếu một Promise thất bại thì toàn bộ thất bại. Dùng khi các tác vụ độc lập với nhau để tiết kiệm thời gian.

Tóm tắt

- Bất đồng bộ giúp chương trình không bị chặn khi chờ tác vụ chậm.
- Promise có ba trạng thái: pending, fulfilled, rejected.
- `async`/`await` kèm `try...catch` cho mã dễ đọc; `Promise.all` chạy song song.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết hàm `cho(ms)` trả về Promise và dùng async/await để in ba dòng cách nhau 1 giây.
2. Dùng `Promise.all` chờ hai tác vụ giả lập cùng lúc và in tổng thời gian.
3. Giải thích sự khác nhau giữa `.then()` và `await`.